

LECTURES 9, 10

BINARY SEARCH TREES

Basic concepts

Directed graph $G = (V, E)$ where:

V is the set of vertices of G , and

E is the set of edges of G .

E is a binary relation on V ; in other words, E is a subset of $V \times V$.

Thus each element of E – that is, each edge – is an ordered pair of vertices.

[Simple examples]

The definition here is that of an directed graph.

In an undirected graph, an edge has to be defined as a set $\{a, b\}$, rather than an ordered pair (a, b) .

Binary search trees are a subtype **TREE**, which is a type of undirected graph.

TREE – an undirected, connected, acyclic graph.

[Simple examples]

Equivalent defining properties of a tree $G = (V, E)$:

1. Any two vertices of G are connected by a unique path.
2. Removal of any one edge of G produces two disconnected components.
3. Addition of any one edge to G produces a cycle in G .
4. G is connected, and $|E| = |V| - 1$.
5. G is acyclic, and $|E| = |V| - 1$.

Rooted tree Basic terms: root node, ancestor, descendant, parent, child, sibling, leaf node, height and depth of a node, height of the tree ...

Ordered rooted tree: the child nodes of any (non-leaf) node are ordered.

Binary search tree: A binary, rooted, ordered tree, in which the following property is satisfied:

Let x be a node in a binary search tree. If y is a node in the left subtree of x , then $\text{key}[y] \leq \text{key}[x]$. If y is a node in the right subtree of x , then $\text{key}[y] \geq \text{key}[x]$.

[Recall that every node has key field and satellite data.]

```
INORDER-TREE-WALK( $x$ )
  If  $x \neq \text{NIL}$  then
    INORDER-TREE-WALK( $\text{left}[x]$ ) ;
    print( $\text{key}[x]$ ) ;
    INORDER-TREE-WALK( $\text{right}[x]$ ) ;
```

Recurrence equation for running time:

$T(n) = T(k) + T(n-k-1) + d$ where k = number of nodes in left subtree

By the simple substitution method, it is shown that $T(n) = \Theta(n)$.

Exercise: Write the recursive algorithms for pre-order and post-order tree walks. Show that their running time is also $\Theta(n)$.

```
TREE-SEARCH( $x, k$ )
  if  $x = \text{NIL}$  or  $k = \text{key}[x]$  then
    return  $x$ ;
  if  $k < \text{key}[x]$  then
    return TREE-SEARCH( $\text{left}[x], k$ ) ;
  else
    return TREE-SEARCH( $\text{right}[x], k$ ) ;
```

Running time $O(h)$, where h is the tree height.

```
ITERATIVE-TREE-SEARCH( $x, k$ )
  while  $x \neq \text{NIL}$  &&  $k \neq \text{key}[x]$ 
    if  $k < \text{key}[x]$  then
       $x = \text{left}[x]$  ;
    else
       $x = \text{right}[x]$  ;
```

We can say that the recursion has been ‘unrolled’ into the while loop.

```

TREE-MINIMUM(x)
  while left[x] <> NIL
    x = left[x];
  return x;

```

```

TREE-MAXIMUM(x)
  while right[x] <> NIL
    x = right[x];
  return x;

```

```

TREE-SUCCESSOR(x)
  if right[x] <> NIL then
    return TREE-MINIMUM(right[x]);
  y = parent[x];
  while y <> NIL && x = right[y]
    x = y;
  y = parent[y];
  return y;

```

If the right subtree of x is not NIL, then the minimum element of that subtree is the successor of x . Otherwise, the successor of x is the lowest ancestor of x whose left child is also an ancestor of x . [→ The lowest ancestor of x in whose left subtree lies x .]

[In a similar way, **TREE-PREDECESSOR**.]

Running time: **TREE-MINIMUM**, **TREE-MAXIMUM**, **TREE-SUCCESSOR** & **TREE-PREDECESSOR** run in $O(h)$ time, where h is tree height.

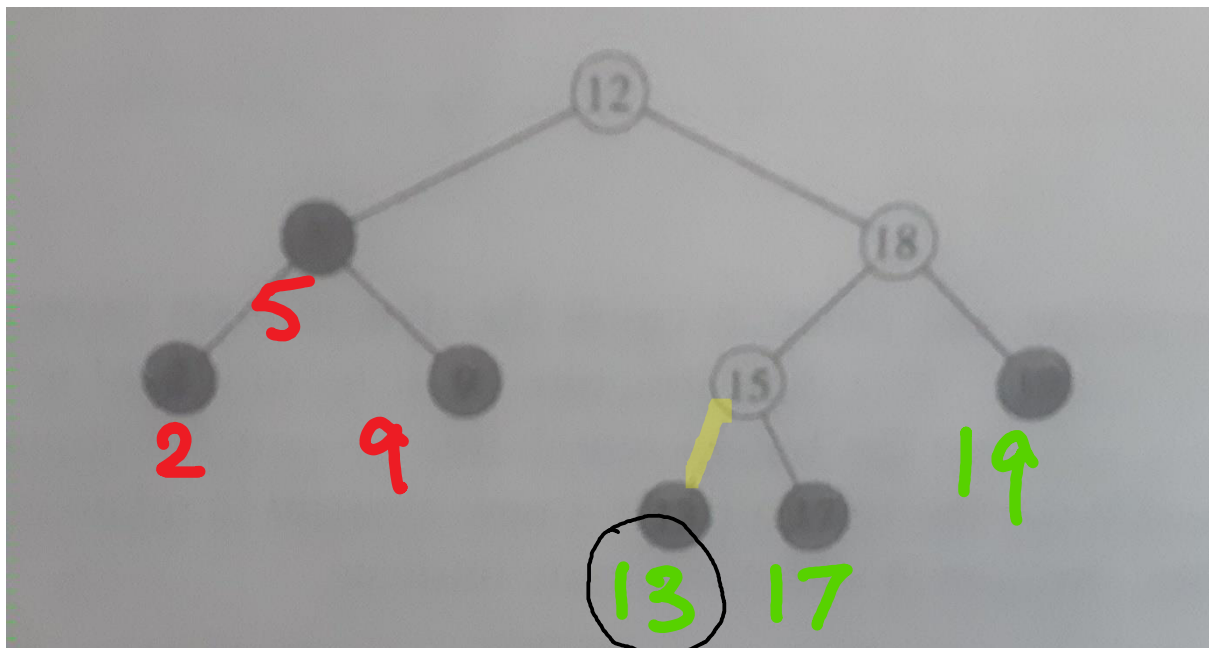
Two important functions are **TREE-INSERT** & **TREE-DELETE**.

```

TREE-INSERT(T, z)      // Note: z is a single node tree
  y = NIL;
  x = root[T];
  while x <> NIL
    y = x;
    if key[z] < key[x] then
      x = left[x];
    else
      x = right[x];
  // found the right place!
  parent[z] = y;
  if y = NIL then
    root[T] = z;      // T was initially empty
  else
    if key[z] < key[y] then
      left[y] = z;
    else
      right[y] = z;

```

The following example is from [CLRS]:



Exercise: Insert into the above tree, by hand-executing the code, a new node with key value 10.

LABORATORY ASSIGNMENTS -- #3 & #4

BINARY SEARCH TREES

1. Using the random number generator, generate $N = 500$ integer key values, as in the previous laboratory assignments.
2. Generate a binary search tree from the above 500 key values, by implementing TREE-INSERT for every incoming node. Determine the height of the tree generated.
3. SORT the 500 key values using a sort program written by you earlier. Generate another binary search tree from these SORTED key values. Determine the height of the tree generated. How does this height compare with the height determined in Step 2?
4. Your class has 100+ students, whose ID number & name data will be provided. Implement a simple database of students using a binary search tree. Insert the student data into the binary search tree. [NOTE: ID Number is key, and name is satellite data.]
5. Write a search function to find the student name from a given ID number.
6. **OPTIONAL – FOR THOSE WHO FINISH EARLY:** (a) Program to show the tree structure, using indentation. (b) Deletion of a record. (c) Given (partial) name, find ID number of student ... *et cetera*.